SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA5101

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Scenario-Based : 20%

Total Marks: 50

Annexure A

Code of Conduct:

I **S. Madhuri (192465036)** ***certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.***

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **S. Madhuri**

Annexure A

1. Secure Password Storage in Web Applications
A web application stores user passwords and needs to protect them against database breaches.

A. Explain why storing plaintext passwords is insecure.
B. Suggest a secure method for storing passwords and justify your choice.

2. Digital Signature for Document Authentication
A company wants to ensure that official documents sent electronically are authentic and unaltered.

A. Explain how digital signatures can be used in this scenario.
B. Justify how this method ensures both integrity and non-repudiation.

3. Preventing Replay Attacks in Online Transactions
An online payment system is vulnerable to replay attacks where attackers resend previously captured transactions.

A. Explain how replay attacks occur.
B. Suggest a cryptographic technique to prevent such attacks and justify your answer.

4. Secure Communication over Public Wi-Fi
A user accesses sensitive information using a public Wi-Fi network without encryption.

A. Identify the risks involved in this scenario.
B. Recommend a secure communication method and justify how it protects user data.

5. Data Integrity in Cloud Storage
An organization stores critical data in a cloud server and wants to ensure that the data is not altered.

A. Explain how hashing can be used to verify data integrity.
B. Suggest an additional mechanism to enhance trust and justify your answer.

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1: Secure Password Storage in Web Applications

A. Why is storing plaintext passwords insecure?

Plaintext storage means saving a user's actual password exactly as entered, without converting it into a protected representation. If an attacker gains access to the database through a breach, stolen administrator credentials, a vulnerable application, or an exposed backup, the attacker can immediately read every stored password. This can cause serious damage because many people reuse passwords across email, banking, social media, and other services.

The main risks are:

- Direct exposure: attackers can use the passwords immediately without cracking anything.
- Credential reuse: one breached password may unlock the victim's accounts on other websites.
- Insider misuse: staff or administrators with database access may be able to view users' passwords.
- Loss of trust and legal consequences: exposed credentials can lead to account takeover, financial loss, and reputational damage.
- Passwords in logs, backups, or debugging output can create additional copies that are difficult to control.

B. Secure storage method and justification

The recommended method is to store a salted password hash using a password-hashing algorithm designed to resist guessing attacks, preferably Argon2id. Suitable alternatives where Argon2id is unavailable include scrypt or bcrypt, configured with an appropriate work factor. A general-purpose hash such as plain SHA-256 is not a good password-storage choice because it is extremely fast and allows attackers to test many guesses per second.

Secure registration and login process:

- When a user registers, the application accepts the password over HTTPS.
- A password-hashing library generates a unique random salt and computes the password hash using Argon2id with a suitable memory cost, time cost, and parallelism.
- The database stores the algorithm and cost parameters, salt, and resulting hash—not the plaintext password.
- During login, the application hashes the submitted password using the stored salt and parameters and safely verifies the result using the library's verification function.

- If the algorithm or cost parameters become outdated, the application upgrades the hash after a successful login.

A salt is not a secret; it makes identical passwords produce different stored hashes and prevents attackers from efficiently reusing precomputed tables. The algorithm's configurable cost makes each guess more expensive. For highly sensitive systems, rate limiting, multi-factor authentication, secure password-reset flows, and breach monitoring should be added.

Decision and justification: Use Argon2id through a reputable library, with parameters benchmarked on the production environment. Never encrypt passwords for later recovery and never store plaintext passwords. Password hashing is intentionally one-way, so the system verifies a password rather than decrypting it.

Conclusion

Salted, deliberately expensive password hashing substantially reduces the damage from a database breach. HTTPS, access controls, rate limits, and multi-factor authentication provide additional layers of protection.

Question 2: Digital Signature for Document Authentication

A. How can digital signatures be used?

A digital signature allows a recipient to check that an electronic document was signed using the holder's private key and that the signed content has not changed. It uses public-key cryptography together with a cryptographic hash.

Signing and verification process:

- The company prepares the final document and calculates a cryptographic hash, such as SHA-256.
- The authorized signer's digital-signature software signs the document hash using the signer's private key and a secure signature algorithm, such as RSA-PSS or ECDSA.
- The company sends the document, signature, and relevant public-key certificate to the recipient.
- The recipient validates the certificate and its trust chain, checks that it identifies the expected signer, and verifies that the certificate is valid for the intended purpose.
- The recipient uses the signer's public key to verify the signature against the received document. If the document has been altered, verification fails.

B. How does this ensure integrity and non-repudiation?

Integrity: the signature is tied to the document's hash. Even a small change to the document normally produces a different hash, causing signature verification to fail. This helps the recipient detect unauthorized changes.

Authentication: a valid signature, together with a properly validated certificate and trusted identity-binding process, provides evidence that the signature was made using the private key associated with the named signer.

Non-repudiation: a valid signature can provide strong evidence that the corresponding private key was used to sign the document. However, it is not an absolute guarantee by itself. The private key must be under the signer's control, protected from theft, and linked to the signer through reliable identity verification. Audit logs, timestamps, certificate status checks, and organizational procedures strengthen the evidence.

Decision and justification: use a recognized digital-signature standard, protect the private key in a secure keystore or hardware security module, validate certificates, and retain the signed document and relevant verification evidence. Encryption is not required merely to create a signature; a signature provides authenticity and integrity, while encryption separately provides confidentiality.

Conclusion

Digital signatures help organizations verify document integrity and signer authenticity. With secure private-key control and trusted certificate validation, they also support strong evidence for non-repudiation.

Question 3: Preventing Replay Attacks in Online Transactions

A. How do replay attacks occur?

A replay attack happens when an attacker captures a valid message and sends the same message again later. For example, an attacker on a compromised device or network might capture an authenticated payment request and resend it. If the server only checks that the request has a valid signature or MAC but does not check whether it has already been processed, the payment could be executed more than once.

A replay attack does not necessarily require the attacker to decrypt or modify the message. The original message may still have a valid cryptographic authentication tag or signature, which is why freshness and duplicate detection are essential.

B. Cryptographic technique to prevent replay attacks

Use a combination of a unique transaction identifier or nonce, a timestamp or expiry window, and server-side replay detection. For high-value financial transactions, the authenticated message should bind all important fields—such as sender, recipient, amount, currency, transaction ID, and expiry—to a digital signature or message authentication code (MAC).

Recommended process:

- The client creates a unique transaction ID or unpredictable nonce and includes it with the transaction details and expiry time.

- The complete message is authenticated with a MAC using a secret shared with the server, or signed using the client's private key where appropriate.
- The server verifies the authentication tag or signature, checks the expiry time, and confirms that the transaction ID or nonce has not been used before.
- The server atomically records the transaction ID as consumed and processes the transaction only once. Repeated requests with the same ID are rejected or return the original result without charging again.
- The system uses TLS to protect the connection and logs transaction IDs and outcomes for audit.

A nonce is a value intended for one-time use. A timestamp alone is insufficient if the same message can be replayed within the allowed time window. A server-side uniqueness check or replay cache is therefore needed. A monotonic sequence number can also work when the server tracks the expected sequence for each client.

Decision and justification: for a payment service, combine TLS, authenticated transaction contents, unique transaction IDs, short expiry windows, and atomic idempotency/replay checks. This prevents an attacker from successfully reusing a previously valid request. The server must perform duplicate detection reliably under concurrent requests.

Conclusion

Cryptographic authentication proves that a message is valid, but freshness controls prove that it is new. Nonces or unique IDs plus server-side duplicate detection are central protections against replay attacks.

Question 4: Secure Communication over Public Wi-Fi

A. Risks of using public Wi-Fi without encryption

Public Wi-Fi may be operated by unknown parties or may contain malicious access points that imitate legitimate networks. If an application sends sensitive data without encryption and authentication, nearby attackers or a malicious hotspot may be able to observe or alter traffic.

- Eavesdropping: passwords, messages, personal information, and session data may be exposed when sent in plaintext.
- Man-in-the-middle attacks: an attacker may intercept communication and attempt to modify messages or impersonate a service.
- Credential and session theft: captured login details or session tokens can enable account takeover.
- Fake hotspot risk: an attacker may create a network with a familiar name to trick users into connecting.
- Malicious redirection and phishing: unprotected traffic may be redirected to fraudulent websites.
- Privacy loss: browsing activity and metadata may be observed even when some content is encrypted.

B. Recommended secure communication method

Use HTTPS with correctly configured TLS for all sensitive web communication. The user should check that the browser is connected to the correct domain and should not ignore certificate warnings. A trusted VPN can add an encrypted tunnel between the device and the VPN provider, but it does not replace HTTPS and does not protect against phishing or a compromised endpoint.

How TLS protects the user:

- Confidentiality: encryption prevents ordinary network observers from reading the protected application data.
- Integrity: authenticated encryption detects unauthorized changes to protected traffic.
- Server authentication: certificate validation helps confirm that the user is communicating with the intended website rather than an impostor.
- Key establishment: the TLS handshake establishes session keys that protect subsequent communication.

Additional precautions include disabling automatic connection to unknown hotspots, keeping the operating system and browser updated, enabling multi-factor authentication, using a password manager, and avoiding sensitive activity if the connection appears suspicious. A VPN can be useful on untrusted networks, but users should choose a trustworthy provider because the VPN operator may see some metadata.

Decision and justification: HTTPS/TLS is the primary recommendation because it protects the application connection end-to-end between the browser and the authenticated website. A VPN is an optional additional layer for broader network traffic protection, not a substitute for valid TLS certificates.

Conclusion

Using correctly validated HTTPS/TLS greatly reduces eavesdropping, tampering, and impersonation risks on public Wi-Fi. Safe network habits and multi-factor authentication further reduce exposure.

Question 5: Data Integrity in Cloud Storage

A. How hashing verifies data integrity

A cryptographic hash function, such as SHA-256, converts data of any supported length into a fixed-length digest. When a file is uploaded, the organization calculates its digest and stores that digest in a trusted location. Later, it downloads or reads the file, calculates the digest again, and compares the result with the trusted original digest.

- Calculate SHA-256 for the original file before upload.
- Store the digest and file version, timestamp, and identifier in a protected integrity record.

- After upload or during a later audit, calculate SHA-256 over the retrieved file bytes.
- Compare the new digest with the trusted stored digest. If they differ, the data may have changed or been corrupted and should be investigated.

A cryptographic hash is useful because even a small data change normally changes the digest. However, a plain hash does not prove who created the file or who stored the reference digest. If an attacker can modify both the cloud file and the digest, the comparison may still pass.

B. Additional mechanism to enhance trust

Use a digital signature over the file's hash and relevant metadata, or use a keyed HMAC when the organization and verifier share a protected secret. A digital signature is particularly useful when independent parties need to verify integrity without sharing a secret key.

With a digital signature, the organization signs a record containing the file hash, file identifier, version, and timestamp using a protected private key. The verifier checks the signature using the trusted public key and independently calculates the file's hash. This protects the integrity record against undetected replacement, provided the signing key and public-key trust process are secure.

Additional controls can include:

- Enable cloud versioning and immutable or write-once retention for critical records.
- Use access controls and multi-factor authentication to limit who can modify files or integrity metadata.
- Keep signed audit logs and monitor unexpected changes.
- Protect signing keys in a managed key service or hardware security module and rotate them under a defined policy.
- Use independent backups and periodically perform restore and integrity-verification tests.

Decision and justification: calculate SHA-256 for routine integrity checks and digitally sign the hash-plus-metadata record to provide authenticity and stronger trust. Use versioning, immutable retention, and audit logs to improve recovery and accountability. Hashing alone detects changes only when the reference digest is trustworthy.

Conclusion

Hash comparison can reveal accidental corruption or unauthorized modification when the reference digest is protected. Digital signatures, secure key management, immutable versions, and audit trails strengthen trust and make tampering harder to conceal.